UNIVERSIDAD DE LAS FUERZAS ARMADAS “ESPE”

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis y Diseño de Software

Taller Grupal

Estudiante:

Lucas Góngora

Klever Jami

Jordy Viscaino

Docente:

Ing. JENNY ALEXANDRA RUIZ ROBALINO

NRC: 30723

Fecha de Entrega:

28 de abril de 2026

ANTECEDENTES....3
OBJETIVO....3
DESARROLLO....3
1. Requisitos Funcionales del Sistema....3
2. Diagrama de casos de uso....4
CASOS DE USO....4
PUENTE....5
3. Matriz de trazabilidad....5
4. Casos de uso....6
1. Requisito Funcional 01 (RF-01)....6
Código PlantUML (Diagrama Individual CU-01)....6
2. Requisito Funcional 02 (RF-02)....7
Código PlantUML (Diagrama Individual CU-02)....7
3. Requisito Funcional 03 (RF-03)....8
Código PlantUML (Diagrama Individual CU-03)....8
4. Requisito Funcional 04 (RF-04)....9
Código PlantUML (Diagrama Individual CU-04)....9
5. Requisito Funcional 05 (RF-05)....10
Código PlantUML (Diagrama Individual CU-05)....10
5. Clases de análisis....11
6. Secuencias de análisis....13
Secuencia: Agregar Estudiante....13
Secuencia: Actualizar Estudiante....13
Secuencia: Eliminar Estudiante....14
Secuencia: Mostrar todos los Estudiantes....15
7. Trazabilidad....16
8. Conclusiones....17
9. Recomendaciones....18
Anexos....19
img 1: Agregar estudiante....19
img 2: Actualizar estudiante....20
img 3: Eliminar estudiante....20
img 4: Estudiante eliminado....21

ANTECEDENTES

Se abordó el análisis funcional de un sistema de gestión académica, donde se identificaron los requisitos funcionales de nivel 0, centrados en las operaciones básicas de creación, lectura, actualización y eliminación, además de la validación de datos. Estos requerimientos dieron pie a un modelos de Casos de Uso, estableciendo las bases de interacción entre el Administrador Académico y el sistema.

A partir de la Guía 2, el trabajo toma como punto de partida los casos de uso para llevarlos hacia un análisis estructural. Aplicando patrones como BCE-R y el uso de la herramienta PlantUML, se demostró cómo los objetos del sistema trabajan entre sí, garantizando que las funcionalidades mantengan coherencia y que la trazabilidad sea correcta desde el requerimiento original hasta la implementación y ejecución.

OBJETIVO

Modelar la estructura y dinámica de un sistema CRUD de estudiantes mediante el uso de diagramas UML, manteniendo y asegurando coherencia y trazabilidad completa entre los requisitos funcionales, los casos de uso, clases de análisis y las secuencias de interacción.

DESARROLLO

1. Requisitos Funcionales del Sistema

RF-01	El sistema debe permitir agregar un estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente.	Administrador académico	Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante existente cuando corresponda.	Administrador académico	Eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados.	Administrador académico	Mostrar todos los estudiantes
RF-05	El sistema debe validar los datos obligatorios antes de agregar o actualizar.	Sistema	Validar datos del estudiante

Requisitos funcionales seleccionados

- **RF-01:** El sistema debe permitir registrar un nuevo estudiante ingresando ID, nombre y edad.
- **RF-02:** El sistema debe permitir modificar la información de un estudiante existente mediante su ID.

- **RF-03:** El sistema debe permitir eliminar un estudiante seleccionado conforme a las reglas académicas establecidas.
- **RF-04:** El sistema debe mostrar el listado completo de estudiantes en una tabla con los campos ID, nombre y edad.
- **RF-05:** El sistema debe validar los datos del estudiante antes de almacenarlos o modificarlos.

Justificación del requisito de validación

El requisito de validación de datos se incorpora como un caso de uso incluido dentro de las operaciones de agregar y actualizar estudiantes porque la verificación de datos es obligatoria para garantizar la integridad de la información.

De acuerdo con la guía, la relación <<include>> debe utilizarse únicamente cuando exista una necesidad funcional real, lo cual se cumple en este caso, ya que el sistema no debe permitir registros con datos inválidos, vacíos o duplicados.

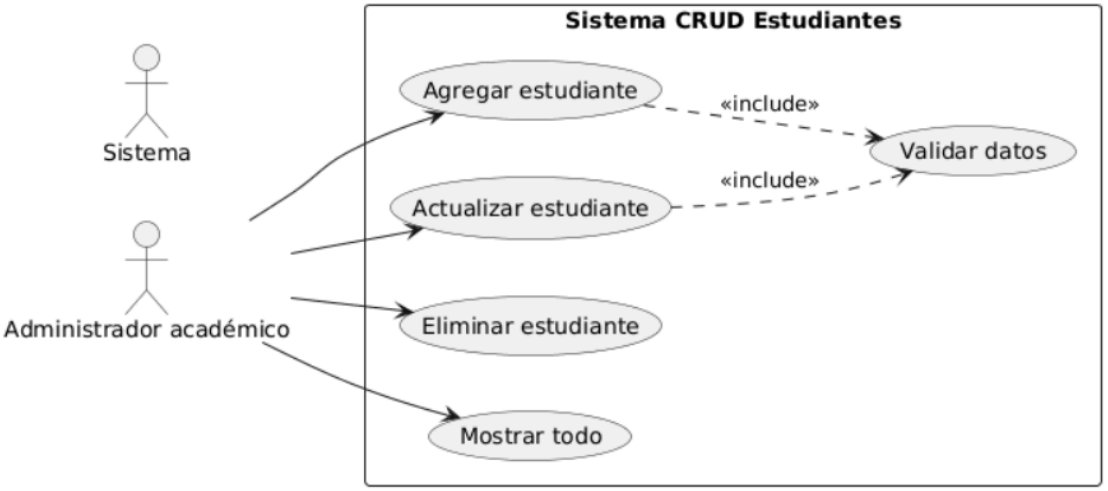
2. **Diagrama de casos de uso**

CASOS DE USO

Código

```
@startuml
left to right direction
actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema CRUD Estudiantes" {
  usecase "Agregar estudiante" as UC1
  usecase "Actualizar estudiante" as UC2
  usecase "Eliminar estudiante" as UC3
  usecase "Mostrar todo" as UC4
  usecase "Validar datos" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

@enduml
```

Resultado

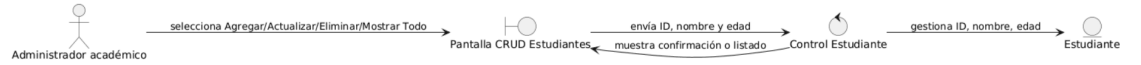


PUENTE

Código:

```
@startuml
 left to right direction

 actor "Administrador académico" as Admin
 boundary "Pantalla CRUD Estudiantes" as UI
 control "Control Estudiante" as Ctrl
 entity "Estudiante" as Est

 Admin --> UI : selecciona Agregar/Actualizar/Eliminar/Mostrar Todo
 UI --> Ctrl : envía ID, nombre y edad
 Ctrl --> Est : gestiona ID, nombre, edad
 Ctrl --> UI : muestra confirmación o listado
 @enduml
```

Resultado



3. **Matriz de trazabilidad**

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia

RF-01	Agregar estudiante	Administrador académico	Alta	El sistema registra un estudiante con ID único, nombre obligatorio y edad válida	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema permite modificar nombre y/o edad validando los datos	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con ID, nombre y edad	UC4
RF-05	Validar datos	Sistema	Alta	El sistema impide guardar datos vacíos o inválidos	UC5

4. Casos de uso

A continuación, se define el Caso de Uso (CU) correspondiente a cada Requisito Funcional (RF) establecido, detallando su propósito y relación para sentar las bases del modelado UML.

1. Requisito Funcional 01 (RF-01)

"El sistema debe permitir registrar un nuevo estudiante ingresando ID, nombre y edad."

Caso de Uso	**CU-01: Agregar Estudiante**
Actor Principal	Administrador Académico
Descripción	Permite al administrador ingresar los datos (ID, Nombre y Edad) de un nuevo estudiante.
Relaciones	<<include>> CU-05: Validar Datos del Estudiante.

Código PlantUML (Diagrama Individual CU-01)

```
@startuml
left to right direction
skinparam packageStyle rectangle
```

```
actor "Administrador Académico" as Admin

rectangle "Sistema CRUD Estudiantes" {
   usecase "Agregar Estudiante" as UC1
   usecase "Validar Datos del Estudiante" as UC5
}

Admin -- UC1
UC1 ..> UC5 : <<include>>
@enduml
```

2. Requisito Funcional 02 (RF-02)

El sistema debe permitir modificar la información de un estudiante existente mediante su ID.

Caso de Uso	**CU-02: Actualizar Estudiante**
Actor Principal	Administrador Académico
Descripción	Permite editar los campos de nombre y edad de un estudiante ya registrado.
Relaciones	<<include>> CU-05: Validar Datos del Estudiante.

Código PlantUML (Diagrama Individual CU-02)

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador Académico" as Admin
```

```
rectangle "Sistema CRUD Estudiantes" {
   usecase "Actualizar Estudiante" as UC2
   usecase "Validar Datos del Estudiante" as UC5
}

Admin -- UC2
UC2 ..> UC5 : <<include>>
@enduml
```

3. Requisito Funcional 03 (RF-03)

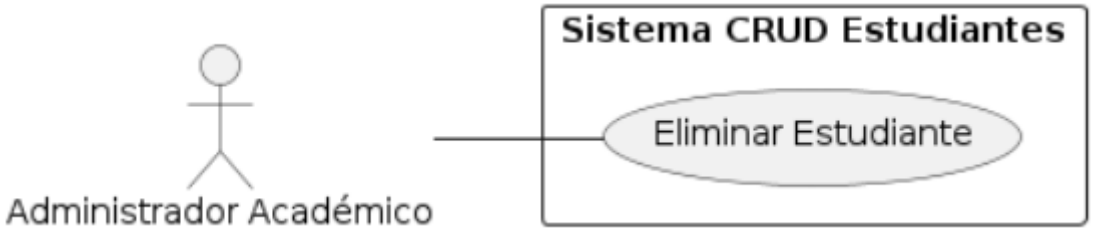
El sistema debe permitir eliminar un estudiante seleccionado conforme a las reglas académicas establecidas.

Caso de Uso	**CU-03: Eliminar Estudiante**
Actor Principal	Administrador Académico
Descripción	Permite remover de forma permanente el registro de un estudiante específico del repositorio.

Código PlantUML (Diagrama Individual CU-03)

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador Académico" as Admin

rectangle "Sistema CRUD Estudiantes" {
   usecase "Eliminar Estudiante" as UC3
}

Admin -- UC3
```

```
@enduml
```



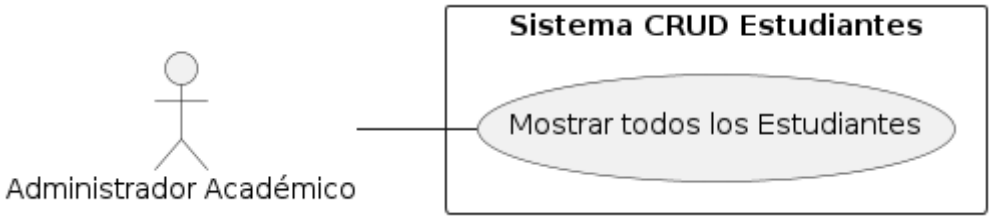
4. Requisito Funcional 04 (RF-04)

"El sistema debe mostrar el listado completo de estudiantes en una tabla con los campos ID, nombre y edad."

Caso de Uso	**CU-04: Mostrar todos los Estudiantes**
Actor Principal	Administrador Académico
Descripción	Genera una vista tabular que recupera y muestra todos los registros de estudiantes almacenados.

Código PlantUML (Diagrama Individual CU-04)

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador Académico" as Admin

rectangle "Sistema CRUD Estudiantes" {
    usecase "Mostrar todos los Estudiantes" as UC4
}

Admin -- UC4
@enduml
```



5. Requisito Funcional 05 (RF-05)

"El sistema debe validar los datos del estudiante antes de almacenarlos o modificarlos."

Caso de Uso	**CU-05: Validar Datos del Estudiante**
Actor Principal	Sistema (Ejecutado internamente)
Descripción	Verifica que el ID, el nombre y la edad ingresados no estén vacíos y cumplan con los formatos esperados.

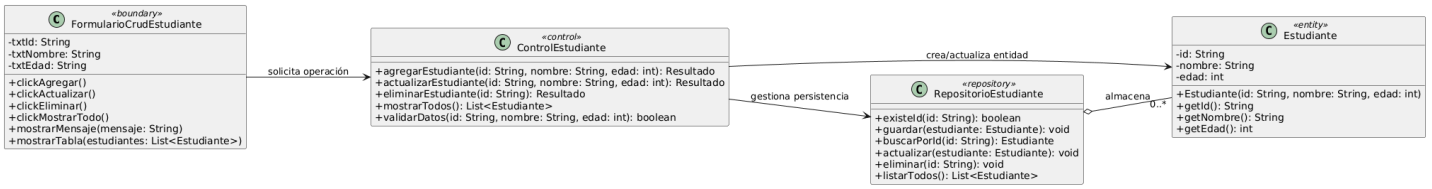
Código PlantUML (Diagrama Individual CU-05)

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Sistema" as Sys

rectangle "Sistema CRUD Estudiantes" {
    usecase "Validar Datos del Estudiante" as UC5
}

Sys -- UC5
@enduml
```

5. Clases de análisis

Para garantizar un diseño robusto, se han identificado las siguientes clases bajo el patrón Frontera-Control-Entidad-Repositorio:

Clase	Estereotipo	Responsabilidad Principal
FormularioEstudiante	<<boundary>>	Interactuar con el Administrador, capturar entradas de datos y desplegar la lista de estudiantes.
ControlEstudiante	<<control>>	Orquestar el flujo de la aplicación, realizar validaciones (RF-05) y decidir cuándo se debe persistir la información.
Estudiante	<<entity>>	Almacenar los atributos básicos del objeto de negocio (ID, Nombre, Edad).
RepositorioEstudiante	<<repository>>	Gestionar la colección de datos, permitiendo las acciones de agregar, buscar, modificar y eliminar físicamente los registros.



Código:

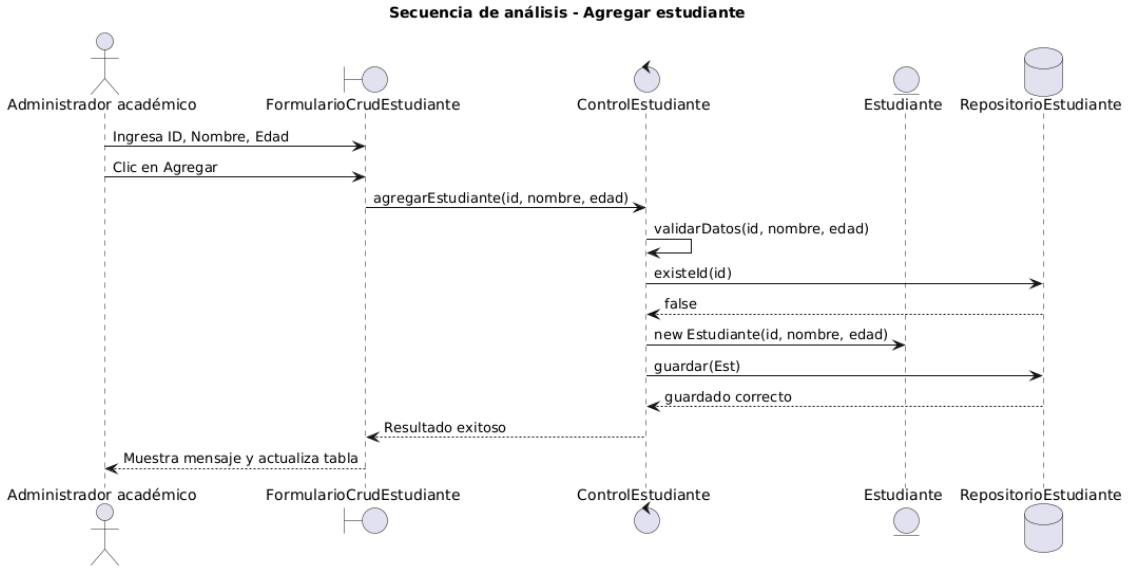
```
@startuml
skinparam classAttributeIconSize 0
```

```
skinparam shadowing false
left to right direction

class "FormularioCrudEstudiante" <<boundary>> {
  - txtId: String
  - txtNombre: String
  - txtEdad: String
  + clickAgregar()
  + clickActualizar()
  + clickEliminar()
  + clickMostrarTodo()
  + mostrarMensaje(mensaje: String)
  + mostrarTabla(estudiantes: List<Estudiante>)
}

class "ControlEstudiante" <<control>> {
  + agregarEstudiante(id: String, nombre: String, edad: int): Resultado
  + actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
  + eliminarEstudiante(id: String): Resultado
  + mostrarTodos(): List<Estudiante>
  + validarDatos(id: String, nombre: String, edad: int): boolean
}

class "Estudiante" <<entity>> {
  - id: String
  - nombre: String
  - edad: int
  + Estudiante(id: String, nombre: String, edad: int)
  + getId(): String
  + getNombre(): String
  + getEdad(): int
}

class "RepositorioEstudiante" <<repository>> {
  + existeId(id: String): boolean
  + guardar(estudiante: Estudiante): void
  + buscarPorId(id: String): Estudiante
  + actualizar(estudiante: Estudiante): void
  + eliminar(id: String): void
  + listarTodos(): List<Estudiante>
}

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
@enduml
```

6. Secuencias de análisis

Secuencia: Agregar Estudiante
Código:

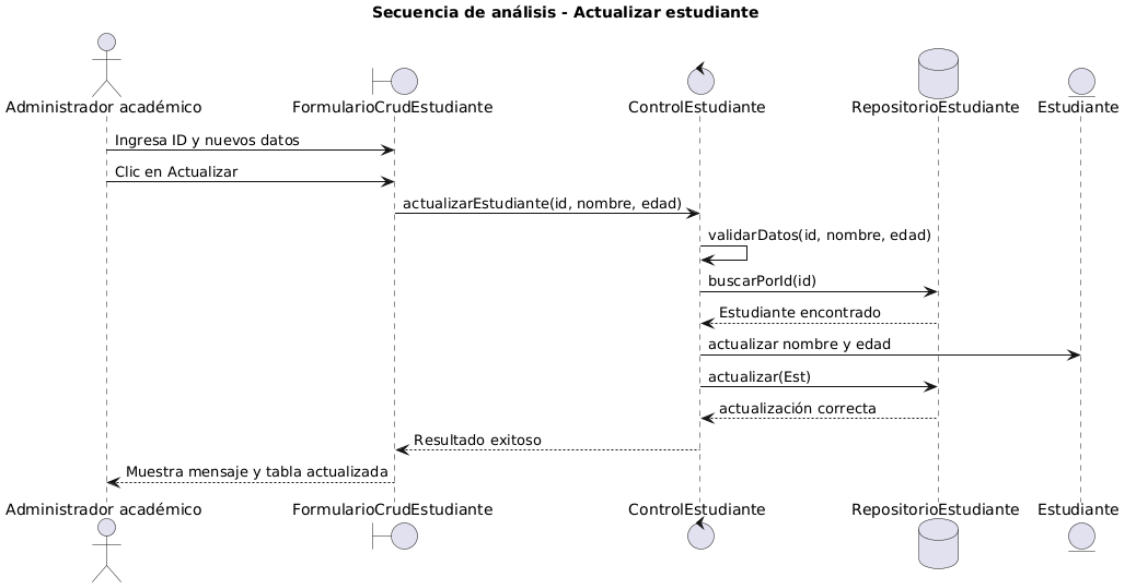
```
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```



Secuencia: Actualizar Estudiante
Código:

```
@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
```

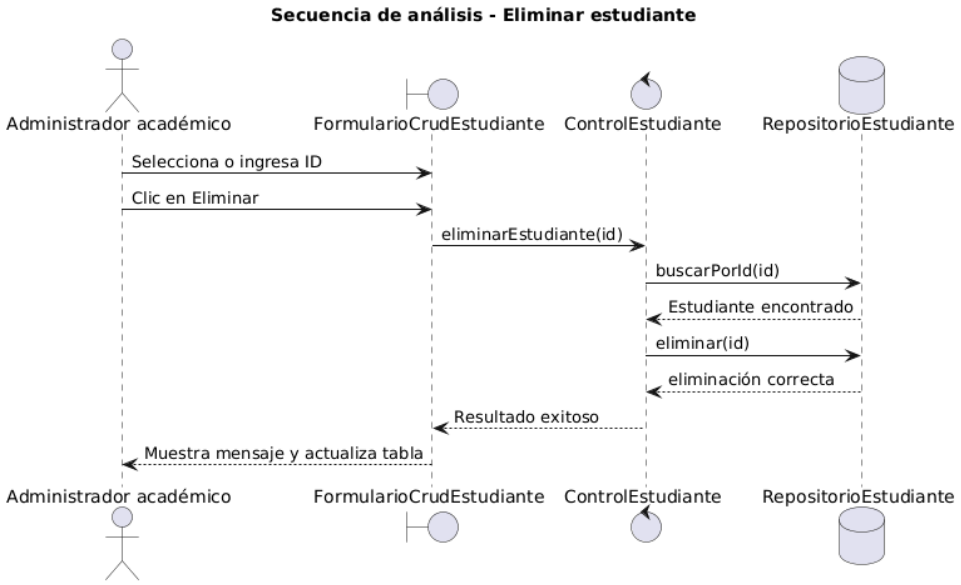
```
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```



Secuencia: Eliminar Estudiante
Código:

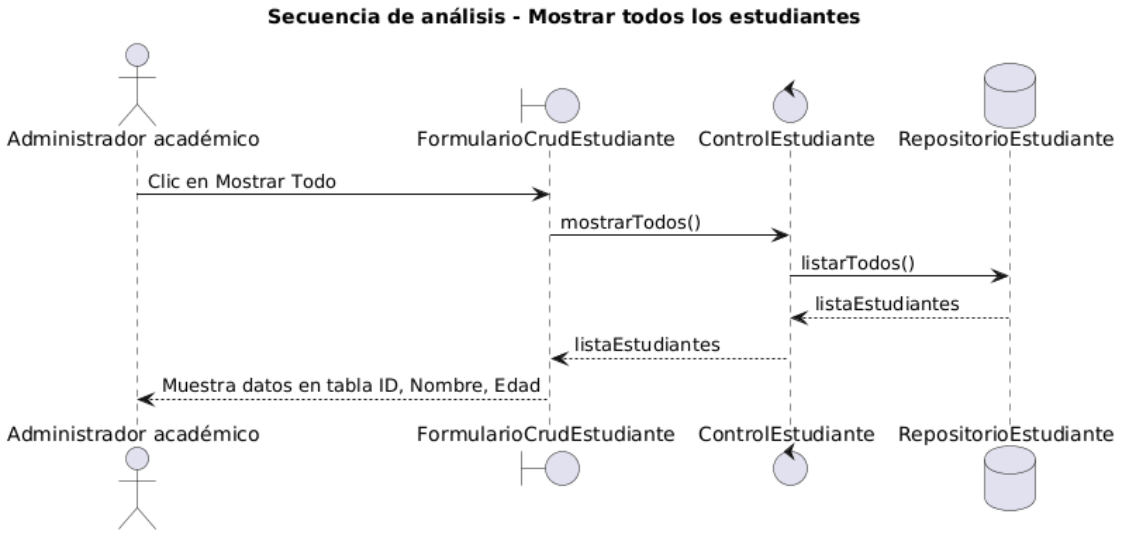
```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
```

```
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```



Secuencia: Mostrar todos los Estudiantes
Código:

```
@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```



7. Trazabilidad

Código RF	Requisito Funcional	Caso de Uso Asociado	Clases de Análisis Involucradas	Diagrama de Secuencia
RF-01	Permitir agregar un estudiante con ID, Nombre y Edad.	Agregar estudiante	FormularioCrudEstudiante, ControlEstudiante, Estudiante, RepositorioEstudiante	Secuencia: Agregar estudiante
RF-02	Permitir actualizar los datos de un estudiante existente.	Actualizar estudiante	FormularioCrudEstudiante, ControlEstudiante, Estudiante, RepositorioEstudiante	Secuencia: Actualizar estudiante
RF-03	Permitir eliminar un estudiante existente.	Eliminar estudiante	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Secuencia: Eliminar estudiante

RF-04	Permitir mostrar todos los estudiantes registrados.	Mostrar todos los estudiantes	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Secuencia: Mostrar todos los estudiantes
RF-05	Validar datos obligatorios antes de agregar o actualizar.	Validar datos del estudiante (Include)	ControlEstudiante	Integrado en secuencias de Agregar y Actualizar

8. Conclusiones

- El sistema cumple con los lineamientos establecidos en la guía al modelar las operaciones principales del CRUD del estudiante: agregar, actualizar, eliminar y mostrar información, además, se incorpora la validación como parte fundamental del proceso, lo que permite mantener consistencia en los datos.

  La relación entre requisitos funcionales, casos de uso y componentes del sistema evidencia un correcto nivel de trazabilidad, lo cual es esencial en el análisis y diseño de software.

- La correspondencia entre los requisitos funcionales y los modelos de análisis garantiza que el sistema responda fielmente a las necesidades del actor mediante una estructura interconectada y coherente.

- El proyecto en Java cumple con la guía 1 porque está basado en un CRUD de estudiante y mantiene trazabilidad entre los requisitos funcionales, los casos de uso y las clases del sistema. La guía pide que el sistema permita agregar, actualizar, eliminar, mostrar y validar datos, y eso coincide con la estructura del código desarrollado.

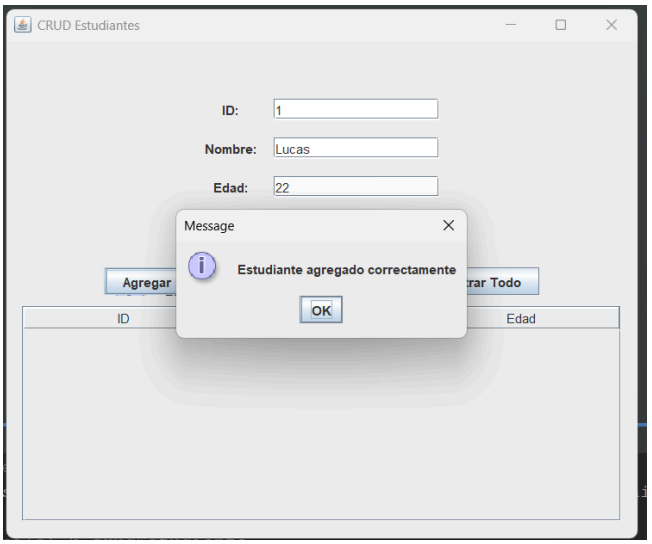
- También se cumple con la idea principal de la guía, que es no hacer un programa por hacer, sino un sistema que pueda justificarse mediante análisis. Cada funcionalidad del código debe relacionarse con un requisito específico, por eso la trazabilidad es importante.

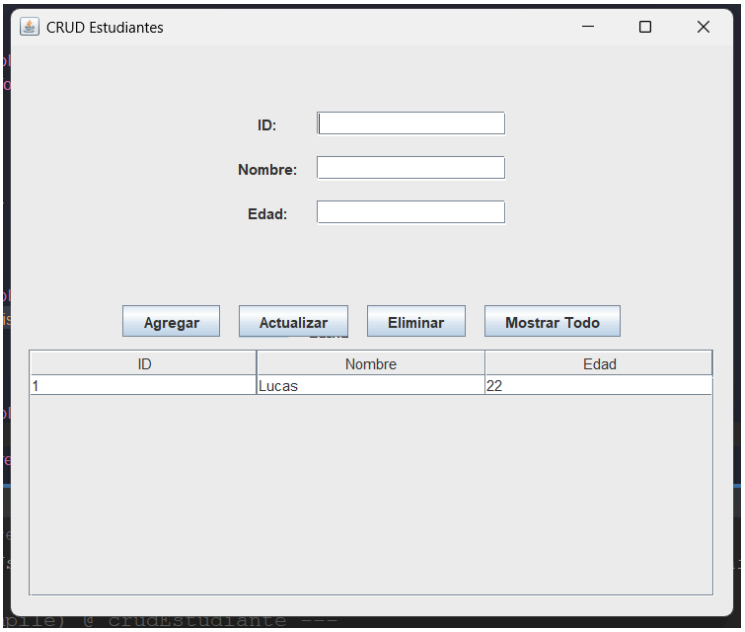
- El sistema puede considerarse correctamente alineado con la guía siempre que se mantenga una relación clara entre los requisitos funcionales, los casos de uso y la implementación. Esto garantiza un diseño coherente, estructurado y verificable.

9. Recomendaciones

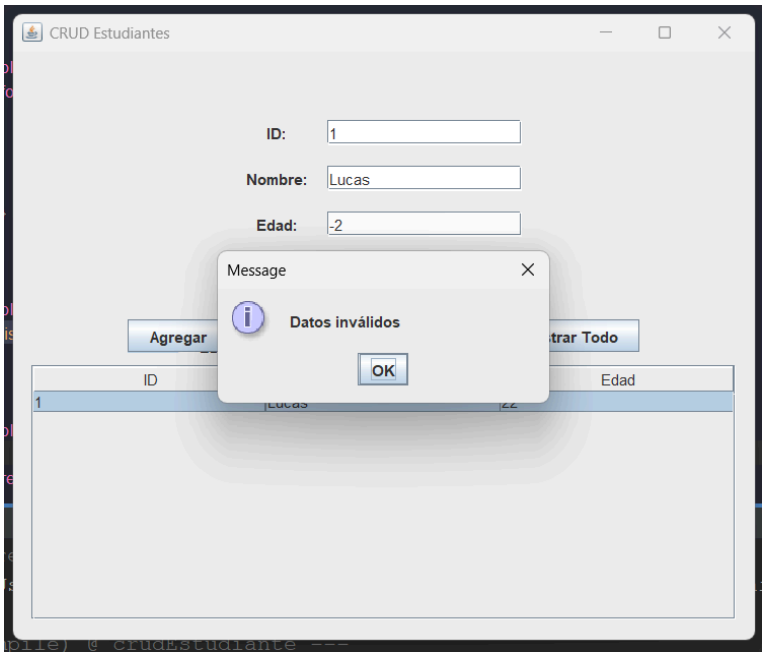
- Verificar que cada mensaje enviado en los diagramas de secuencia corresponda estrictamente a una operación declarada en el diagrama de clases de análisis.
- Mantener una separación clara de responsabilidades utilizando los estereotipos boundary, control y entity, evitando mezclar detalles de la interfaz con la lógica de negocio.
- Asegurar que la validación de datos (RF-05) se ejecute siempre de manera previa a cualquier acción de persistencia o guardado en los flujos de secuencia.
- Garantizar que la entidad mantenga la consistencia de los atributos (ID, Nombre y Edad) en todos los modelos para asegurar la integridad de la información.
- Elaborar un diagrama de secuencia independiente por cada operación principal para evitar la sobrecarga de información y facilitar la lectura del modelo dinámico.
- No se deben incluir funciones que no estén contempladas dentro de los requisitos establecidos.
- Cada método implementado en el código debe estar directamente asociado a un caso de uso específico.
- El documento debe evidenciar la relación existente entre los requisitos, los casos de uso y su implementación en el código.
- Los nombres de las funciones deben ser claros y reflejar el caso de uso que representan.

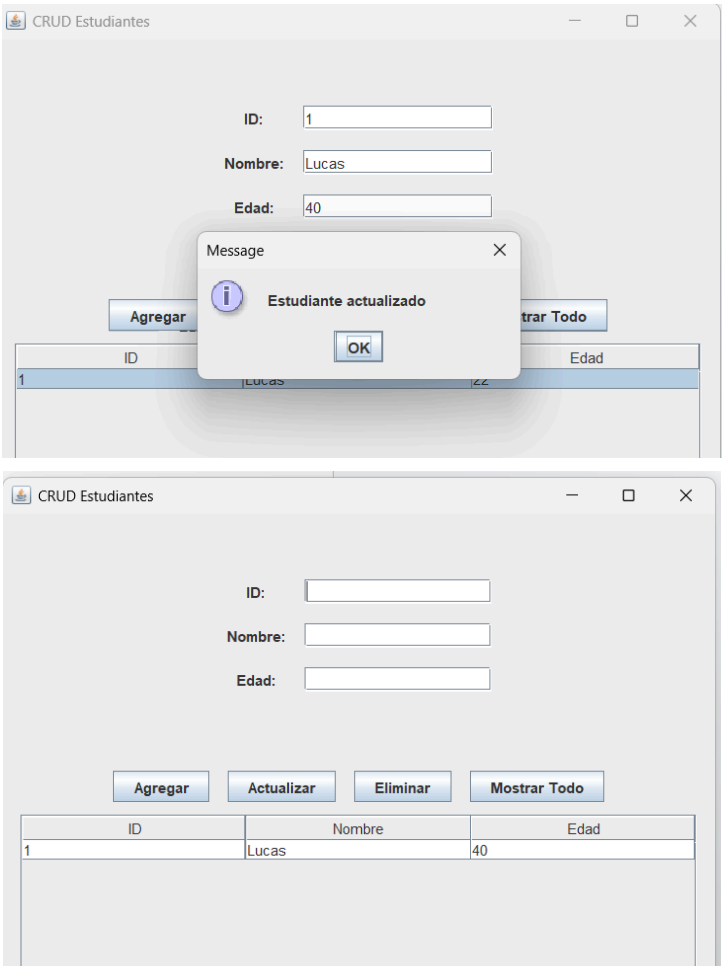
Anexos



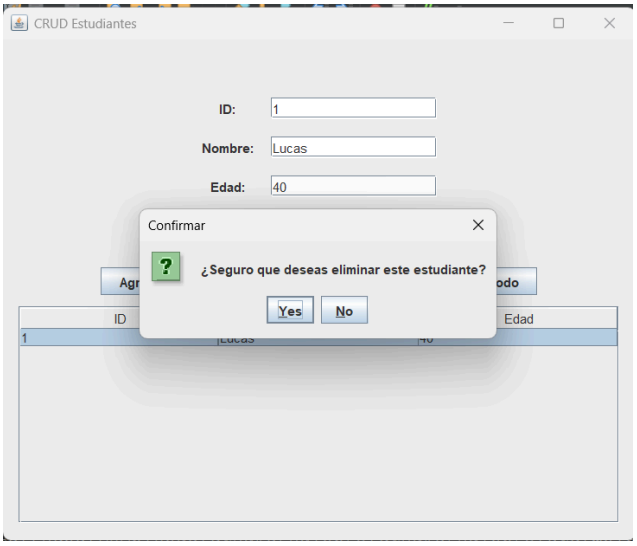


img 1: Agregar estudiante

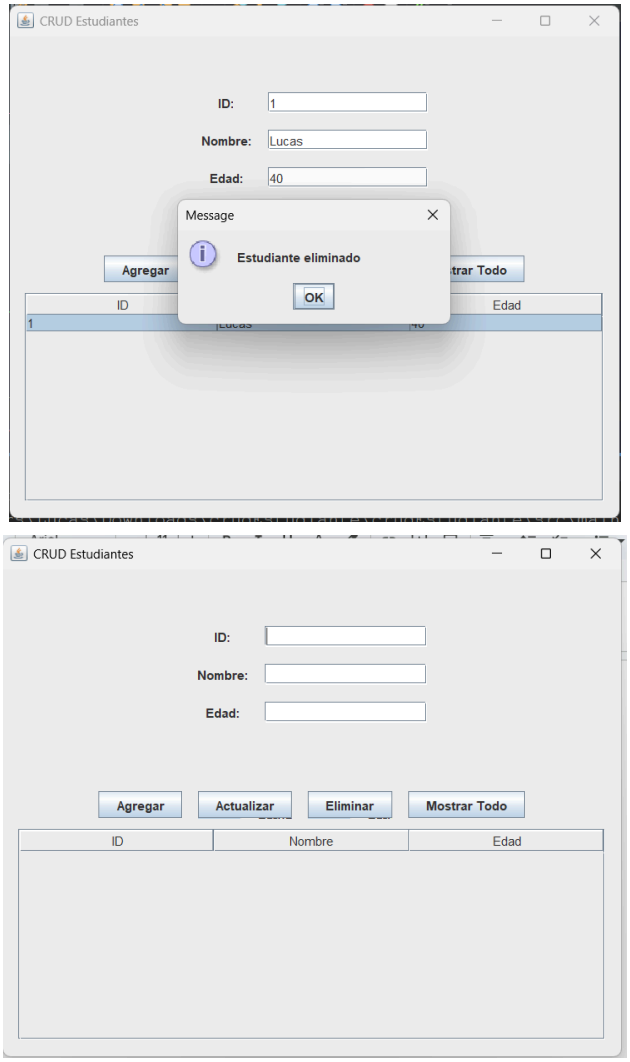




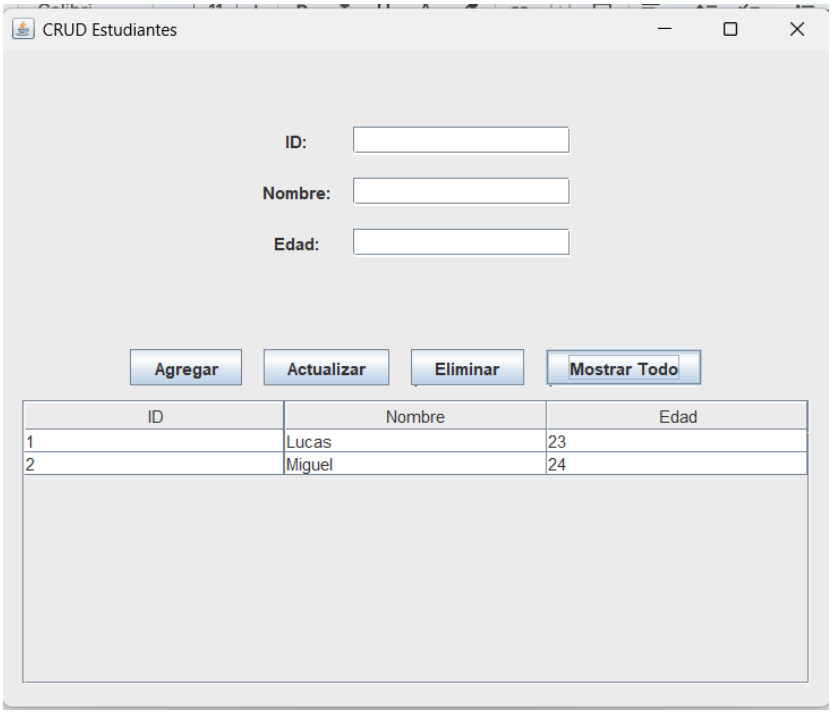
img 2: Actualizar estudiante



img 3: Eliminar estudiante



img 4: Estudiante eliminado



img 5: Mostrar Todo